# Requirements Analysis Document

Sabatino Raffaella

September 8, 2026

**Version:** 0.3
**Version date:** September 8, 2026

## Version History

| Version number | Revision date | Description of change |
|---|---|---|
| 0.1 | September 8, 2026 | Initial version of the requirements analysis document. |
| 0.2 | September 8, 2026 | 1° Version of Introduction (Scope and Project Overview). |
| 0.3 | September 8, 2026 | Defined 1UC, 2UC and updated Constraint and Design Decisions. |

# Contents

# 1 Introduction

## 1.1 Project overview

PriceWatcher was originally conceptualized to check for discounts on mouses with side buttons and find the most economic one while also having something that can endure stress.

I decided to expand on the idea by making it for general product price check and notification of said price change.

## 1.2 Scope

System will be implemented using both an extension and a React webapp that will act as a Dashboard.

Extension is the piece that will allow the user to pin down a product while navigating, it will sit on the side allowing user to view the product name, current price and a way to add the product to the watchlist.

The Dashboard is where the user can keep track of the products inside the watchlist and view price changes.

## 1.3 Brief description of PriceWatcher

At the moment, PriceWatcher consists of these two main components but still requires further refinement.

The main purpose of this document is to keep track of the system's requirements, design decisions, and development progress.

# 2 Use Cases

## 2.1 UC-01 Add product

**Actor:** User.

**Goal:** Add a product to the watchlist so that its price can be monitored and viewed in the dashboard.

**Trigger:** The user opens the PriceWatcher extension panel while viewing a product page.

**Preconditions:**

- The PriceWatcher extension is installed and enabled.
- The user is viewing a product page on a supported website.

**Main Scenario:**

1. The extension retrieves the product name, page URL, current price, and currency from the product page.
2. The extension displays the product name and current price, together with an *Add to watchlist* action.
3. The user reviews the displayed information and selects *Add to watchlist*.
4. The system validates the required product information and checks whether the product is already in the watchlist.
5. The system saves the product in the watchlist and records its current price, currency, and observation time as the initial price record.
6. The extension confirms that the product has been added successfully.
7. The product becomes available in the dashboard's list of monitored products.

**Alternative Scenarios:**

**A1. Unsupported page or missing information.** If the page is unsupported or the required product information cannot be retrieved, the extension explains that the product cannot be added. No watchlist entry is created.

**A2. Product already monitored.** If the product is already in the watchlist, the extension informs the user. No duplicate entry is created.

**A3. Save failure.** If the product cannot be saved, the extension displays an error and allows the user to retry. It does not report a successful addition.

**A4. User cancels.** If the user closes the panel without selecting *Add to watchlist*, the watchlist remains unchanged.

**Postconditions:**

- On success, the watchlist contains one entry for the product, with its identifying information and initial price record.
- The product is available for subsequent price monitoring and display in the dashboard.
- On cancellation or failure, no new product entry is added to the watchlist.

### 2.2 UC-02 Monitor product price

**Actor:** User.

**Goal:** Monitor a watched product's price through the React webapp dashboard and identify changes since the previous successful price check.

**Trigger:** The user selects the dashboard link in the PriceWatcher extension panel to check a watched product's price.

**Preconditions:**

- The PriceWatcher extension is installed and enabled.
- At least one product has been added to the watchlist through UC-01.
- The React webapp dashboard can access the user's watchlist and stored price records.

**Main Scenario:**

1. The user opens the extension panel and selects the link to the dashboard.
2. The system opens the React webapp dashboard and loads the user's watchlist.
3. The user locates the product they want to monitor. The dashboard displays its name, latest recorded price, currency, and the time of the last successful price check.
4. When a price check runs, the system retrieves the product's price from its saved page URL and validates the result.
5. The system compares the retrieved price with the previous recorded price in the same currency to determine whether it has increased, decreased, or remained unchanged.
6. The system saves the valid price observation and its timestamp.
7. The dashboard displays the updated price, the price change, and the time of the successful check, allowing the user to distinguish newly retrieved information from older records.

**Alternative Scenarios:**

**A1. Empty watchlist.** If no products are being monitored, the dashboard displays an empty-state message and explains that the user can add a product through the extension.

**A2. Dashboard or watchlist unavailable.** If the dashboard cannot be opened or the watchlist cannot be loaded, the user is informed of the failure and can retry. Stored products and price records remain unchanged.

**A3. Price retrieval fails.** If the product page is unavailable or a valid price cannot be extracted, the dashboard retains the last successfully recorded price and its timestamp, indicating that the latest check failed. No invalid price observation is saved.

**A4. Price unchanged.** If the retrieved price matches the previous price, the system records the successful check and updates its timestamp without indicating a price increase or decrease.

**A5. Currency differs.** If the retrieved currency differs from the stored currency, the system does not calculate a price change or overwrite the last valid price. The dashboard indicates that the new price could not be compared.

**Postconditions:**

- On success, the product's latest valid price observation and check time are stored and available in the dashboard.
- The user can see whether the price changed relative to the previous comparable observation.
- If the check fails, existing valid records are preserved and are not presented as newly retrieved prices.
- Discount threshold configuration, price notifications, and detailed price history are covered separately by UC-03, UC-04, and UC-06.

## 2.3 UC-03 Set discount threshold

## 2.4 UC-04 Receive price notification

UC-04 — Notify user of significant discount

Actor: User

Trigger: A monitored product reaches the configured discount threshold.

Preconditions: - Product is being monitored. - A target discount has been configured.

Main Scenario: 1. The extension retrieves the current product price. 2. The system calculates the current discount. 3. The system compares the discount with the configured threshold. 4. If the threshold has been reached, the system checks whether this threshold has already generated a notification. 5. If not, the system generates a notification. 6. The system records the notification threshold.

Alternative Scenarios: A1. Discount is below the threshold. → No notification is generated.

A2. Discount reaches a threshold that was already notified. → No duplicate notification is generated.

Postconditions: - The user has been notified, if appropriate. - The notification state is updated.

### 2.5 UC-05 View monitored products

### 2.6 UC-06 View price history

## 3 Use Case Scenarios

### 3.1 UC-01 Add product

**Preconditions**

**Main scenario**

**Alternative scenarios**

**Postconditions**

### 3.2 UC-02 Monitor product

**Preconditions**

**Main scenario**

**Alternative scenarios**

**Postconditions**

## 4 System Requirements

### 4.1 Functional requirements

**FR-01 — Product monitoring**

The system shall allow the user to add a product to the monitored products list.

**FR-02 — Price updates**

The system shall periodically retrieve the current price of monitored products.

**FR-03 — Discount notification**

The system shall notify the user when the product reaches or exceeds the configured discount threshold.

**FR-04 — Notification state**

The system shall avoid repeatedly notifying the user for the same discount threshold.

### 4.2 Non-functional requirements

## 5 Data / Domain Model

### 5.1 Products

### 5.2 Watchlist

### 5.3 Price records

### 5.4 Notifications

## 6 Constraints & Design Decisions

### 6.1 Chrome extension

The system will use a Chrome extension as the primary interface for adding products to the Watchlist while browsing. The extension allows PriceWatcher to interact directly with the page being visited and retrieve the information required to identify and monitor a product.

Chrome extensions were chosen because of their accessibility and their ability to retrieve information from websites while remaining lightweight and easy to use.

### 6.2 React dashboard

The system will implement its Dashboard using React with Vite as the frontend technology stack. React was chosen both because of its widespread adoption in modern frontend development and because the project may eventually serve as a portfolio project.

React's component-based architecture naturally encourages a modular approach to development. During implementation, the ability to structure the Dashboard as a collection of independent and reusable components has made it straightforward to separate functionality and reuse common elements across different parts of the application. Vite also provides a lightweight development environment with straightforward development, testing, and deployment workflows.

### 6.3 Parser architecture

The system will use a Parser architecture to ensure its functionality across different sites.

It will use a main parser which will delegate the parsing process to a site-specific parser based on the site the user is currently visiting.

### 6.4 Storage

As of the current version, PriceWatcher uses Chrome Storage for persistent data storage.

Chrome Storage was chosen because the expected amount of stored data is relatively small and the system is intended for personal use. The number of products monitored by a user is not expected to be large enough to require a dedicated database at this stage.

The storage solution may be reconsidered in the future if the system's scope or data requirements increase, particularly with the introduction of more extensive price history.

## 7 Future Requirements

*User must be able to view price history and changes. User must also be notified if there are any discounts/price drop on watched products.*